

MFA Implementation Guide

OTP Authentication using Twilio Verify

Login (user+pass) → Temp Token → Send OTP → User enters code → Verify OTP → Real Session Token → Access Granted

Phase 1: Twilio Setup

- Step 1** Create a Twilio account at [twilio.com](https://www.twilio.com) and verify your phone number.
- Step 2** Get your **Account SID** and **Auth Token** from the Twilio Console dashboard.
- Step 3** Purchase a Twilio phone number (or use a free trial number).
- Step 4** Enable **Twilio Verify** service — Console → Verify → Services → Create new service.
- Step 5** Note your **Verify Service SID** (starts with [VA...](#)).

Phase 2: Backend Changes

- Step 6** Install the Twilio SDK: `npm install twilio` or `pip install twilio`
- Step 7** Store credentials in environment variables — **never hardcode** them: `TWILIO_ACCOUNT_SID`, `TWILIO_AUTH_TOKEN`, `TWILIO_VERIFY_SID`
- Step 8** Add fields to your users table: **phone_number** (VARCHAR), **mfa_enabled** (BOOLEAN), **mfa_verified** (BOOLEAN)
- Step 9** Create endpoint `POST /auth/send-otp` — triggers Twilio Verify to send OTP to user's phone.
- Step 10** Create endpoint `POST /auth/verify-otp` — validates the OTP code entered by user.
- Step 11** Create endpoint `POST /profile/setup-mfa` — lets users enroll their phone number for MFA.

Phase 3: Login Flow Changes

- Step 12** On successful username/password login, **do not issue a session/token yet**.

Step 13	Return a temporary token (short-lived JWT, ~5 min expiry) indicating 'password verified, MFA pending'.
Step 14	Frontend redirects user to OTP entry screen.
Step 15	Call <code>/auth/send-otp</code> using the temp token — Twilio sends SMS to the registered phone.
Step 16	User enters the 6-digit OTP received via SMS.
Step 17	Call <code>/auth/verify-otp</code> with the entered code.
Step 18	On success, issue the real session token/JWT and complete the login.

Phase 4: OTP Send Logic (Node.js)

```
// Node.js example using Twilio Verify
const client = require('twilio')(accountSid, authToken);

async function sendOtp(phoneNumber) {
  const verification = await client.verify.v2
    .services(verifyServiceSid)
    .verifications.create({ to: phoneNumber, channel: 'sms' });
  return verification.status; // 'pending'
}
```

Phase 5: OTP Verify Logic (Node.js)

```
async function verifyOtp(phoneNumber, code) {
  const result = await client.verify.v2
    .services(verifyServiceSid)
    .verificationChecks.create({ to: phoneNumber, code });
  return result.status === 'approved';
}
```

Phase 6: Security Hardening

Step 19	Rate limit the <code>/auth/send-otp</code> endpoint (e.g., max 3 requests per 15 min per user).
Step 20	OTP expiry — Twilio Verify handles this automatically (10-minute window).

Step 21	Invalidate temp token after OTP verification (success or failure).
Step 22	Lock account after N consecutive failed OTP attempts (e.g., 5 attempts).
Step 23	Log MFA events — send/verify attempts, failures, and IP addresses.

Phase 7: MFA Enrollment Flow (First-Time Setup)

Step 24	After first login (password only), prompt user to enroll a phone number.
Step 25	Send a verification OTP to confirm they own the number.
Step 26	Only set <code>mfa_enabled = true</code> after successful code verification.
Step 27	Generate and show backup codes (8–10 one-time codes) — store hashed in DB.

Phase 8: Frontend Changes

Step 28	Add OTP input screen (6-digit input, ideally auto-focus + auto-submit on last digit).
Step 29	Add 'Resend OTP' button with a 60-second cooldown timer.
Step 30	Handle error states: invalid code, expired code, too many attempts.
Step 31	Add MFA setup screen in user profile / settings.

Phase 9: Edge Cases to Handle

Step 32	Phone number change — re-verify the new number before updating.
Step 33	Lost phone — allow bypass via backup codes.
Step 34	International numbers — store in E.164 format (e.g., +14155552671).
Step 35	Trusted devices — optionally skip MFA for remembered devices via a long-lived cookie.

Phase 10: Testing & Go-Live

- | | |
|--------------------|---|
| Step
36 | Test with Twilio's test credentials and magic numbers to avoid SMS costs during dev. |
| Step
37 | Test failure paths — wrong code, expired code, locked account. |
| Step
38 | Move to production Twilio credentials before go-live. |
| Step
39 | Monitor via Twilio Console logs for delivery failures. |

Tip: Use Twilio Verify (not raw SMS) — it handles OTP generation, expiry, retries, and international delivery automatically.